

Deep Dive — Module 6: Docker & Docker Compose (Infrastructure)

***Goal:** understand how the five services run together reproducibly — images vs containers, port mapping, volumes for persistence, Dockerfiles, Docker Compose for orchestration, and the everyday commands. This is the "glue" that makes `docker-compose up` start the whole system.*

Where it sits: Docker is the **packaging and runtime layer**. Qdrant, PostgreSQL, the embedding service, the backend, and the frontend each run in a container; Docker Compose wires them into one network so they can talk to each other, and gives you a single command to bring the stack up or down. The entire project is on-premise, so this reproducibility is essential.

1. Why containers

A container bundles an app with *everything it needs* — runtime, libraries, config — so it runs identically on a laptop, a test box, or the on-prem server. No more "works on my machine." For a five-service system with two databases, this consistency is the difference between a 10-minute setup and a week of dependency debugging.

2. Images vs containers

Image = the blueprint / recipe (read-only, versioned)
Container = a running instance of an image (the cooked meal)

One image → many containers. Stopping a container doesn't delete its image.

```
docker pull qdrant/qdrant           # download an image from a registry
docker run qdrant/qdrant             # run a container (foreground)
docker run -d qdrant/qdrant          # run detached (background)
docker run -d--name qdrant -p 6333:6333 qdrant/qdrant # named + port-mapped
```

3. Port mapping

`-p HOST:CONTAINER` exposes a container's internal port on your machine. Without it, the container is unreachable from outside.

```
laptop:6333 → qdrant:6333      laptop:5432 → postgres:5432
laptop:5000 → embedding:5000   laptop:8080 → backend:8080   laptop:3000 → frontend:3000
```

*Inside a Compose network, services reach each other by **service name** (e.g. `http://embedding-service:5000`), not `localhost`. `localhost` inside a container means that container, not the host.*

4. Volumes (persistent data)

By default, data written inside a container is **lost when it's removed**. A **volume** stores data on the host so it survives restarts — essential for PostgreSQL.

```
docker run -d--name postgres \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \   # named volume holds the DB files
  -p 5432:5432 postgres:15-alpine
```

The financial database *must* use a volume; Qdrant likewise persists its vectors and snapshots to a volume so an index rebuild isn't needed after every restart.

5. Dockerfile (building your own image)

A `Dockerfile` is the recipe for building a custom image — used here for the Python embedding service and the backend/frontend.

```
FROM python:3.10-slim          # base image
WORKDIR /app                   # working dir inside the container
COPY requirements.txt .         # copy dependency manifest first (layer caching)
RUN pip install -r requirements.txt
COPY embed_service.py .        # then copy app code
EXPOSE 5000                     # document the port
CMD ["python", "embed_service.py"] # what runs on start
```

```
docker build -t my-embedding-service .
docker run -p 5000:5000 my-embedding-service
```

Layer-caching tip: copy `requirements.txt` and install before copying source. Then code changes don't bust the dependency layer, so rebuilds are fast.

6. Docker Compose (orchestration)

Compose defines all services in one YAML file and starts them together with networking and volumes handled for you.

```

version: '3.8'
services:
  qdrant:
    image: qdrant/qdrant:latest
    ports: ["6333:6333"]
    volumes: ["qdrant_data:/qdrant/storage"]
    restart: unless-stopped

  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: ${DB_PASSWORD}    # from .env, not hard-coded
      POSTGRES_DB: financial_docs
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
    restart: unless-stopped

  embedding-service:
    build: ./embedding-service             # build from local Dockerfile
    ports: ["5000:5000"]
    restart: unless-stopped

volumes:
  pgdata:
  qdrant_data:

```

- `image:` pulls a pre-built image; `build:` builds from a local Dockerfile.
 - `environment:` injects config; secrets come from a `.env` file or the host, never committed.
 - `restart: unless-stopped` auto-recovers crashed containers.
 - `depends_on` (and healthchecks) can control startup order so the backend waits for Postgres/Qdrant.
-

7. Everyday commands

```
docker-compose up -d          # start all services in background
docker-compose down           # stop & remove containers (keeps volumes)
docker-compose down -v        # also delete volumes (DESTROYS all data!)
docker ps                     # list running containers
docker-compose logs -f qdrant # follow live logs for one service
docker-compose restart postgres # restart one service
docker-compose build           # rebuild images after code changes
```

down -v is the dangerous one — it wipes the financial database. Know the difference between `down` and `down -v`.

8. Security & ops notes (project-specific)

- **Pin images to digests** (not just `:latest`) so builds are reproducible and supply-chain-safe (OWASP A05/A08).
- **Don't expose Qdrant/Postgres ports to the internet** — only the backend reaches them on the internal network; publish only what's needed.
- **Secrets via env/ `.env` /vault**, never baked into images or committed.
- **Health checks** per service feed the backend's `/health` aggregation and Compose's startup ordering.
- **Resource sizing**: the embedding model + databases want a decent host (the PRD assumes ~8-core / 16 GB RAM).

9. Common pitfalls

- **localhost inside a container** — refers to the container itself; use the service name on the Compose network.
- **No volume on Postgres/Qdrant** — data vanishes on `down` or container recreation.
- **down -v by habit** — silently destroys the database.
- **Using `:latest` everywhere** — non-reproducible builds; pin versions/digests.

- **Copying source before installing deps** — defeats layer caching, slow rebuilds.
 - **Hard-coded secrets in the Dockerfile/compose file** — leak risk; externalize them.
 - **Publishing every port** — exposes internal services unnecessarily.
-

10. Practice exercises

Install Docker Desktop / Engine first.

1. `docker run -d--name qdrant -p 6333:6333 qdrant/qdrant` ; hit `http://localhost:6333` and confirm it responds.
 2. Run Postgres with a named volume; create a table, `docker-compose down` , bring it back, and confirm the table persists.
 3. Repeat without a volume and observe the data loss.
 4. Write a Dockerfile for the Flask embedding service; build and run it; `curl /health` .
 5. Write a `docker-compose.yml` with qdrant + postgres + embedding-service; `docker-compose up -d` and `docker ps` .
 6. From inside one container (`docker exec -it.. sh`), reach another by **service name** (e.g. `curl http://embedding-service:5000/health`).
 7. Tail logs of one service with `logs -f` ; restart it and watch the output.
 8. Practice the difference: `down` then `up` (data kept) vs `down -v` then `up` (data gone).
-

11. Self-check questions

- What's the difference between an image and a container?
- What does `-p 6333:6333` do, and why is it needed?
- Why must PostgreSQL use a volume in this project?
- Inside Compose, how does the backend reach the embedding service — and why not `localhost` ?
- What's the difference between `image:` and `build:` in Compose?
- Why pin image versions instead of using `:latest` ?
- What does `down -v` destroy, and how is it different from `down` ?
- Where should secrets live, and where must they not?

12. Glossary

- **Image** — read-only app blueprint.
 - **Container** — running instance of an image.
 - **Port mapping** — `HOST:CONTAINER` exposure.
 - **Volume** — host-backed persistent storage.
 - **Dockerfile** — recipe to build an image.
 - **Docker Compose** — multi-service orchestration via one YAML.
 - **Service name** — DNS name a container uses to reach another in the same network.
 - **Registry** — where images are stored/pulled (e.g. Docker Hub).
-

Navigation: [← Module 5 Python/Embeddings](#) | [Index](#) | [Module 7 Tesseract OCR](#) →